Group 36 Final Project Report: Machine Learning-Based Weather Prediction

Jamie Easterbrook, Ahmed Rashrash, David Zhong
{easterbj,rashrasa,zhongx22}@mcmaster.ca

1 Introduction

Our challenge is to construct a model that will predict immediate short-term weather labels around Toronto, given a large sample of past weather information. There has been a lot of recent development with AI-based weather models, and there is no shortage of large, open-source weather datasets to train models with. Because of the boom in AI models across various fields, including weather prediction, and many developments later, newer ones have been performing significantly better than traditional weather prediction methods, such as physics-based models, in both accuracy and speed (Rackow et al., 2024).

Our approach to solving this problem is with a recurrent neural network (RNN), which can make use of the sequential nature of weather patterns. Additionally, we looked transformer models, which are good at interpreting sequential data in parallel. Our implementation will be based on simplified architecture for a RNN-based multi-headed attention system (Vaswani et al., 2023). These models can often outperform most other RNNs, even with relatively worse parameters, so it is a good alternative to our base RNN (Willard et al., 2025).

Many aspects of weather prediction are heavily time-based so a good model would be able to capture such patterns. A good example of this is that when close to a large lake, the chance of rain increases after a period of high temperatures due to Lake-Effect precipitation (Miner and Fritsch, 1997). Since purely data-driven models can capture patterns like this without needing the background reasoning of why or how it happens, it follows that they are overtaking physics based-models with recent developments (Rackow et al., 2024). Data-based models are much faster and more efficient to run than numerical models, but can't handle extreme points in the data nearly as well, resulting in rare weather conditions being poorly handled (Waqas et al., 2025), so we will need to weight our models accordingly to account for this.

2 Related Work

One project we discovered involved generating accurate predictive data for several weather metrics (Han et al., 2021). This project was a neural network-based predictive algorithm, with several iterations of this model. The project compared feed-forward neural networks (FNNs) to recurrent networks, with one of the RNN implementations being the gated recurrent unit, and that aimed to solve the vanishing gradient problem, which was being encountered. The models each took sequential weather data and used it to predict future data points, in order to make clean, more consistent synthetic data for other models to draw on.

Another project we looked into was an RNN-based weather prediction task that would classify images based on non-exclusive weather states (Zhao et al., 2019). They used self-annotated data, and their model would return a list of probabilities that each state was occurring. This model initially was based on a single variable classifier, but the paper found that a multi-label approach was the most efficient and accurate way to label outputs. This makes intuitive sense, as weather conditions often overlap, and reducing them to a single causes significant loss of information.

3 Dataset

Our dataset is a large sample of public domain weather data taken from the Government of Canada's website through a provided Bash script. The dataset consists of measurements taken roughly every 3 hours, ranging from June 11^{th}, 2013, to September 26^{th}, 2025. The exact time between measurements is highly irregular, resulting in inconsistent amounts of labels on a day-to-day basis. Additionally, the table is full of null values, and

many empty or extraneous columns, including two that were completely unlabeled, one of which being the crucial and highly predictive precipitation amount.

Due to the frequency and time range of recordings, our dataset has 107,000 data points, which means we can afford to drop invalid or incomplete points.

3.1 Labels

The table is provided as a .csv file, with built-in labels for each point, classifying what weather conditions were currently unfolding at the given time. These labels are grouped together and treat different magnitudes of the same weather as different prefix labels, for example “Mostly Cloudy” and "Partially Cloudy” are treated as separate cases.

We ended up simplifying and combining many of these redundant class labels, dropping our list of 27 classes down to only 9 in a new, simplified dataset. We trained models on both the initial class list, and the simplified class list to see how lowering the number of labels affected accuracy metrics. This would also eliminate a potential issue of having mutually exclusive labels appear together (sunny and overcast, for example), while also helping discover more patterns about the dataset itself.

3.2 Preprocessing

In preprocessing, missing/redundant values were removed from the dataset. Since this model is designed to predict weather labels for a specific non-changing location, features such as the latitude and longitude are constant and irrelevant. For example, the raw weather data included some station-designated flags for certain values (indicating that certain values were estimated or missing), and a humidex column, but these were largely empty and the entire columns were dropped.

Additionally, 3 new features were created from the Date/Time values, which were then removed (except for the year). The feature "dtHours", was created to keep track of the number of hours since the previous measurement that ended up in the final dataset. This was important since RNN models (the first one we experimented with) typically are designed under the assumption that each data instance in the sequence is equally apart from the previous and next instance. The other two features, “sinTime" and sinDay", are the time in hours and day of the year values passed through an appropriate periodic function in order to capture the periodic nature of these features. Day 1 and day 365 of the year are closer together than day 1 and 5, and it was important to capture this detail. Similar was done for the time. The periods for sinTime and sinDay were 24 and 365, respectively.

Lastly, each label set (comma-separated) was split into individual labels, all unique labels were identified, and a column was created for each unique label. If a label belonged to a datapoint, its column was filled with a 1, while the rest were filled with zeroes. The final transformed dataset was then used for later splitting.

4 Features

The features of our processed dataset include:

- Temp (°C)
- Dew Point Temp (°C)
- Wind Chill
- Wind Speed (km/h)
- Wind Direction (10s of degrees)
- Air Pressure (kPa)
- Visibility (km)
- Relative Humidity (%)
- Time since last measurement (Hours)
- Time (Sinusoidal Embedding)
- Day (Sinusoidal Embedding)
- Current Year

Our input values contain all of this information, as we have filtered out any columns missing any values. These features were selected by manual inspection, choosing the most complete columns.

We engineered the date and time features to have more readable numeric values to be more compatible with our RNN. We added in a delta time column, corresponding to the time in hours since the last measurement. Additionally, due to the periodic nature of the hours in a day and the days in a year (days 1 and 364 are close together), these were converted into a sinusoidal representation with periods 24 and 365 respectively.

5 Implementation

For each of our two model implementations, we trained them on both the full dataset and the simplified version with only 9 classes. These simplified models were created to check whether reducing the number of classes would meaningfully reduce the false positive rate, which was the main tripping point we ran into with our initial tests. As shown in Figure 1, both the simplified models outperformed the higher dimensional models in terms of total

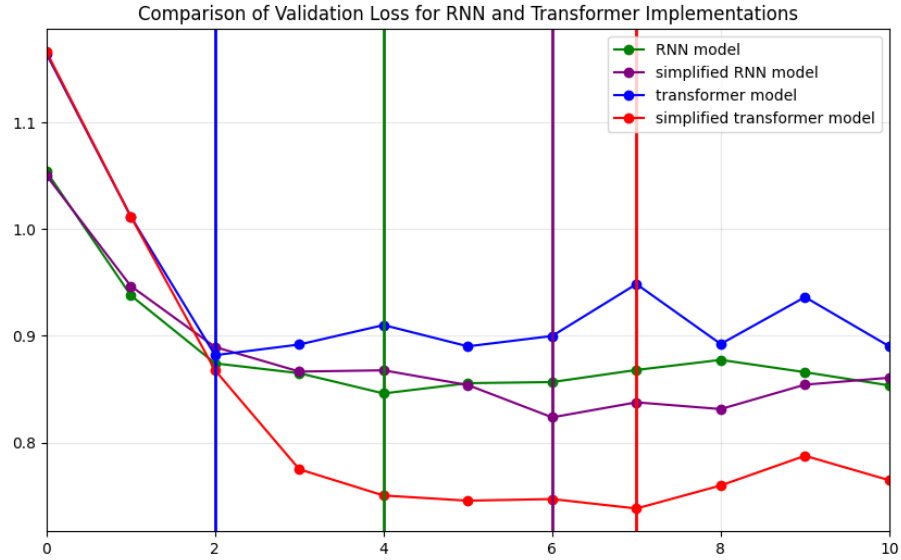

Figure 1: Analysis of the loss functions of our four model implementations (calculated with Binary Cross-Entropy)

loss, and the simplified transformer model had the best loss curve by far, with the least amount of overfitting.

5.1 Recurrent Neural Network

Our first model is a recurrent neural network (RNN), implemented through PyTorch's built-in RNN module. A (vanilla) RNN was selected because it's the simplest version in the family of RNNs.

The output of the RNN is followed by a fully connected layer mapping the final hidden layer to the output, which isn't activated at this point. The input layer accepts 12 input features, an input sequence of 300, and a batch size of 32.

All features are normalized through the per-feature mean and standard deviation, on the training and validation sets. The means and standard deviations were derived solely from the training data, and then applied to the validation data after. This was done to prevent data leakage since the validation set is meant to be treated as unseen data for which the per-feature means and standard deviations are unknown.

A sequence length of 300 was chosen because on average, datapoints are 3 hours apart. However, this isn't always the case. Some data points are 1 hour apart and some hundreds of hours apart. This extra padding helps ensure we have more data than what may have been necessary, which results in better or equally good predictions than with too little data.

Since this is a multi-label classification problem, the final activations are eventually to be computed through the sigmoid function, not softmax. Additionally, there are 3 stacked RNN layers, 256 hidden neurons per layer, and some regularization (L2 and dropout) in place.

The intended purpose of constructing a deeper learning model with 3 stacked layers is so that the model learns both short and long term patterns. By extension, the intended purpose of having 256 neurons per hidden layer is so that the model has the capacity to learn more complicated patterns, should they exist. In short, these decisions were made primarily to avoid prior beliefs about the complexity of the problem constraining the model's peak learning capacity.

For regularization, we analyzed how the model performs with and without both L2 regularization and dropout, based on the validation loss, accuracy, recall, precision, and the F1 score, of the different models. We found that applying both led to the best results, with the smoothest curve and the lowest amount of overfitting. This became the norm moving forward for all future models in this report.

The loss function module used was PyTorch's `BCEWithLogitsLoss`, which combines binary cross entropy loss with the sigmoid function, and takes advantage of a mathematical simplification to provide better numerical stability. The sigmoid implied with this loss function is the reason why the neural network doesn't have an activation at the end. When calculating validation loss, the sigmoid of the final linear layer is calculated manually.

The optimizer we used was PyTorch's `Adam` op-

timizer, or Adaptive Moment Estimation. This adjusts the learning rate of the parameters as the model is training, by using a weighted average of past gradients. This reduces oscillations in the dataset, allowing for faster convergence, especially with larger datasets.

Lastly, because this is a multi-label classification problem with 27 possibilities and at most 3 labels per instance, the target set is dominated by zeroes. The `BCEWithLogitsLoss` loss function provided by PyTorch allows you to specify a 'pos_weight' parameter which allows you to increase the loss of false negatives by adding weights for each of the targets (27 weights in this case). These weights were set to be the inverse of the frequency of each label, making rarer labels, such as "Thunderstorms" (target 27), more costly to miss.

5.2 Transformer

For our second model, we implemented a transformer neural network model, based on a simplified version of a multi-headed attention mechanism built atop existing neural network architecture (Vaswani et al., 2023). This model was implemented using PyTorch's encoder functions, and is similar in construction to the RNN we implemented. We kept many of the same design choices as in the RNN for this transformer model, as we wanted a fair comparison between the models. We will detail the major differences between the two implementations.

Compared to our RNN, our transformer model uses the same sequence length, number of hidden neurons, layer count, initial learning rate, dropout rate, and batch size. We kept many of the hyperparameters the same, since given the similarity between the two models the reasoning for picking those values still holds for a transformer implementation.

Since this function uses a multi-headed attention, works by splitting the hidden layers into multiple parts and running the attention mechanism on each part in parallel. This allows it to perform different transformations with each head, and then recombine these values to capture more intricate relationships. For this implementation, we decided on using 8 heads, and since the hidden layer is equal to 256, this means each head performs their attention mechanism in 32-dimensional space.

Our loss function is PyTorch's `BCEWithLogitsLoss`, which was still the best option to ensure consistency while training. Additionally, keeping the loss functions the same between the two models ensures that loss graphs will show us a comparable training progression.

The chosen optimizer was PyTorch's `AdamW`, which separates weight decay from the optimization process, making it a more effective regularization method. This speeds up convergence of the training, which is useful as training a transformer model is very computationally expensive, so minimizing the required number of epochs is very important.

Alongside `AdamW`, an LR scheduler, `LambdaLR` was implemented, to help correct and guide the adaptive learning rate from `AdamW`. For our purposes, using both in conjunction is helpful for our large dataset, and ensures the best results while training.

6 Results

We performed a comparison of accuracy, recall, precision, and F1 score between our four models and some rough baselines for our multivariate classification. The baselines we chose were:

- **All-Zero Baseline:** Always predicts absence (0) for every weather event.
- **All-One Baseline:** Always predicts presence (1) for every weather event.
- **Majority Vote Baseline:** Always predicts 1 for the most common class(es) and 0 for others.

Additionally, we analyzed how the models perform against metrics from a similar RNN classification model (Zhao et al., 2019). The two models we chose to compare to were a standard RNN with a long short-term memory (LSTM), which was the most similar to our RNN, and another iteration that includes an attention mechanism, which is similar to our transformer model.

From our results in Table 1, we see that all models outperform our All Zeros and All Ones baselines in terms of F1 score. However only the simplified models can outperform the majority vote, as they have far better F1 scores due to the higher precision values.

When we compare to similar models, we see that the recall scores are roughly similar, meaning that all our models are just as good at identifying positive class labels. However the poor precision scores compared to both our comparison models indicate a high amount of false positives with our correct predictions. This means that our model is including

Method	Accuracy	Recall	Precision	F1 Score
Model (RNN)	84.49%	77.12%	16.96%	27.81%
Model (Simplified RNN)	78.17%	86.52%	32.94%	47.71%
Model (Transformer)	82.16%	81.04%	15.71%	26.32%
Model (Simplified Transformer)	80.61%	84.34%	35.57%	50.03%
Baseline (All Zeros)	96.12%	0.00%	0.00%	0.00%
Baseline (All Ones)	3.87%	100.0%	3.87%	7.45%
Baseline (Majority Vote)	94.63%	28.57%	29.88%	29.20%
Comparison (CNN with LSTM)	–	87.39%	74.58%	80.48%
Comparison (CNN with Attention)	–	84.72%	86.43%	85.57%

Table 1: Evaluation metrics on the test set. Comparison models are from (Zhao et al., 2019).

extraneous class labels, which is lessened by the simplified dataset, hence the better performance of the simplified models.

7 Evaluation

The dataset was split into training, validation and testing with an 81-9-10 split ratio. First, 10% of the data was reserved for testing purposes, while 90% of it was kept for the main dataset. Then the main dataset was further split, with 81% used for training and 9% for validation. All splits kept temporal ordering without shuffling to preserve the nature of time series. The validation dataset was used to determine the correct point for early-stopping.

We didn't use the traditional cross-validation, because a naive k-fold approach would leak the future information for forward-looking prediction like a weather model. Researchers have proposed techniques like "rolling window" to employ cross-validation for time series (Bergmeir and Benítez, 2012). However, due to time constraints, we didn't implement it in this project.

We used accuracy, recall and precision and F1 score as the metrics for evaluation, the same as in the progress report. They identified the pattern in the models' behaviour well: Both had a high accuracy and recall (over 80%) but low precision.. This indicated they followed a conservative prediction strategy; often giving predictions of the most common labels, resulting in many false positives.

8 Progress

Consistent with our plan, we explored additional climate datasets to increase the amount of data and improve its overall quality. One dataset we found useful was provided by the National Oceanic and Atmospheric Administration (NOAA). Despite its rich features and reliable records, it was not used in the final model due to the challenge of making U.S. data compatible with previously collected Canadian data. To address label sparsity in the Toronto Pearson Airport dataset, we merged similar weather conditions to consolidate rare categories, which helped the model achieve better performance and faster training when trained on this simplified dataset. Also in line with our plan, we built a Transformer model that is more sophisticated than the RNN model we implemented in the progress report. Compared to traditional RNNs, the Transformer architecture uses multi-headed self-attention to capture global relationships across the entire input sequence at once. This made it more effective for climate-related time-series data, as expected. However, we continued to use purely numeric features as input, without incorporating satellite images as the TA suggested. This choice was made due to the cost and inefficiency of training graphical data, especially given our limited computational resources.

9 Error Analysis

During early iterations of the RNN model, a big issue was that the model was learning to simply predict all zeros. Because there were 27 possible outputs, one for each possible label, and with at most 3 labels in a sample (this was multi-label classification), the dataset targets were filled with zeros. This caused the model to learn that its best bet was to simply predict zero no matter what (initially every prediction was a zero) and this resulted in a deceptively high accuracy percentage. After implementing a larger penalty for false negatives, which was proportional to how rare the class was, this problem was significantly reduced.

However, since false negatives were being penalized, the number of false positives started to rise. This initially appeared to be simply just a lack of

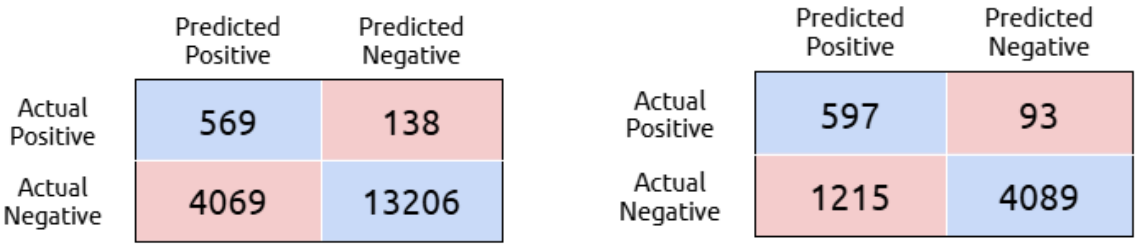

Figure 2: Confusion Matrices of Initial RNN (Left) and Simplified RNN (Right)

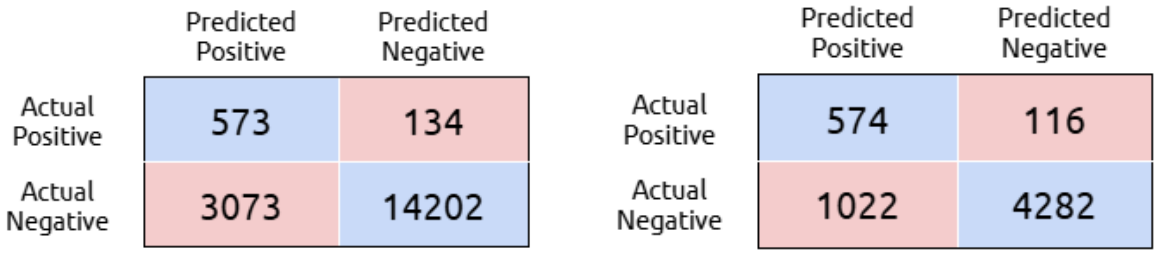

Figure 3: Confusion Matrices of Initial Transformer (Left) and Simplified Transformer (Right)

fine-tuning while solving the previous issue, but this started making less sense as a closer look was given to the predictions. The model started predicting common classes such as "Clear" and "Cloudy" more often. While the accuracy metric was falling because of false positives, the predictions themselves didn't seem too off-base. Some of the labels in the original dataset were incomplete. For some samples, one could reasonably infer the missing labels. For example, most of the samples which had a variant of the "Rain" or "Rain Shower" labels didn't have a variant of the "Cloudy" label. In the real world, rain is rarely not accompanied by significant cloud coverage. For this reason, targeting higher recall values was prioritized over precision or accuracy. In other words, it was more important to identify weather events when they should be, than to correctly predict that nothing was happening.

As shown by Figure 2 and Figure 3, dimensionality reduction improved every metric. This is no surprise, however, as the model is doing less (9 outputs instead of 27), and the granular labels such as "Mostly Cloudy" have been replaced for more general ones (e.g. simply "Cloudy") encompassing multiple possible labels. This is more realistic to expect from a smaller model trained on incomplete data.

As shown by Table 1, our base transformer model performs worse than our base RNN in terms of Accuracy, Precision, and F1 score, but has slightly better recall. This is most likely due to errors with training, as the validation loss shown by Figure 1 indicates almost immediate overfitting, and very few epochs worth of training. This is most likely due to poor training parameters, as it had a large batch size and a high initial learning rate, even before the scheduler modified it. This model was extremely computationally expensive to train, taking upwards of 10-15 minutes per epoch even with this setup, and sometimes even causing problems with the computer running the model if the hyperparameters were any larger. This hardware limitation was a large source of error for this model, but it was fixed by simplifying the input features, which leads to the simplified transformer having the smoothest and smallest validation loss curve, and the best performance for any of our models.

10 Team Contributions

Ahmed Rashrash:

- Initial plan
- Sections 5.1 and 9
- Pre-processing and feature engineering
- RNN implementation, partial hyperparameter tuning
- Error analysis and confusion matrices for both models
- Reviewed spelling, grammar, wording, accuracy/correctness

Jamie Easterbrook:

- Sections 1-4, 5.2, 6
- LaTeXformatting
- Analysis of mean-squared error and comparison metrics
- Research on past works for comparison
- Transformer implementation, evaluation

David Zhong:

- Sections 7 and 8
- Data Processing
- Explored training models on Google Cloud

References

Christoph Bergmeir and José M. Benítez. 2012. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213. Data Mining for Software Trustworthiness.

Jung Min Han, Yu Qian Ang, Ali Malkawi, and Holly W. Samuelson. 2021. Using recurrent neural networks for localized weather prediction with combined use of public airport data and on-site measurements. *Building and Environment*, 192:107601.

Todd J. Miner and J. M. Fritsch. 1997. Lake-effect rain events. *Monthly Weather Review*, 125(12):3231 – 3248.

Thomas Rackow, Nikolay Koldunov, Christian Lessig, Irina Sandu, Mihai Alexe, Matthew Chantry, Mariana Clare, Jesper Dramsch, Florian Pappenberger, Xabier Pedruzo-Bagazgoitia, Steffen Tietsche, and Thomas Jung. 2024. Robustness of ai-based weather forecasts in a changing climate. *Preprint*, arXiv:2409.18529.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2023. Attention is all you need. *Preprint*, arXiv:1706.03762.

Muhammad Waqas, Usa Wannasingha Humphries, Bunthid Chueasa, and Angkool Wangwongchai. 2025. Artificial intelligence and numerical weather prediction models: A technical survey. *Natural Hazards Research*, 5(2):306–320.

Jared D. Willard, Peter Harrington, Shashank Subramanian, Ankur Mahesh, Travis A. O'Brien, and William D. Collins. 2025. Analyzing and exploring training recipes for large-scale transformer-based weather prediction. *Artificial Intelligence for the Earth Systems*, 4(2):240061.

Bin Zhao, Xuelong Li, Xiaoqiang Lu, and Zhigang Wang. 2019. A cnn-rnn architecture for multi-label weather recognition. *Preprint*, arXiv:1904.10709.